CASE STUDY 1: UNIVERSITY SCHOLORSHIP ALLOCATION

1.Problem statement

To develop a Python program to check scholarship eligibility based on aggregate marks, attendance, and annual family income.

2.objective

To check students' aggregate marks.

To verify attendance percentage.

To check annual family income.

To determine scholarship eligibility.

3.Assumption

Minimum marks: 80%.

Minimum attendance: 75%.

Maximum annual family income: Rs. 2,00,000.

All conditions must be satisfied.

4.Algorithm

1. Start.

2. Read marks, attendance, and income.

3. Check whether marks are at least 80%.

4. Check whether attendance is at least 75%.

5. Check whether income is at most Rs. 2,00,000.

6.If all conditions are satisfied, display scholarship approved.

7.Otherwise, display scholarship not approved.

8.Stop.

5.Flow chart

START

|

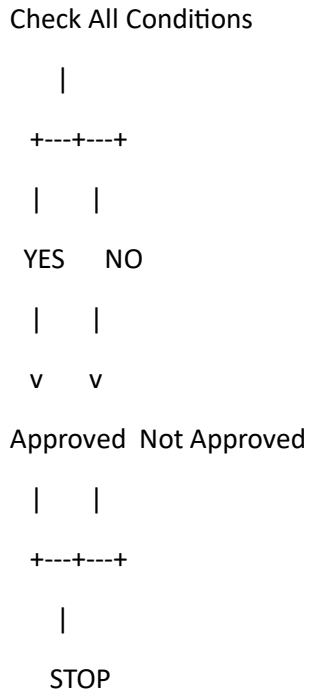
v

Input Marks,

Attendance, Income

|

v



6.Pseudocode

```
BEGIN
INPUT marks, attendance, income

IF marks >= 80 AND attendance >= 75
AND income <= 200000 THEN
   PRINT "Scholarship Approved"
ELSE
   PRINT "Scholarship Not Approved"
END IF

END
```

7.Python source code

```
marks = int(input("Enter aggregate marks: "))
attendance = int(input("Enter attendance percentage: "))
income = int(input("Enter annual family income: "))

if marks >= 80 and attendance >= 75 and income <= 200000:
   print("Scholarship Approved")
```

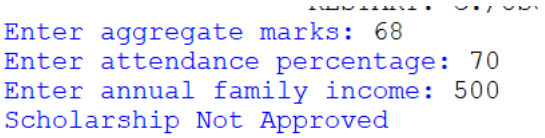
else:

print("Scholarship Not Approved")

8.Test cases

Marks	Attendance	Income	Expected Output
90	85	150000	Approved
75	85	150000	Not Approved
90	65	150000	Not Approved
90	85	250000	Not Approved

9.Out put screen shot



10.Conclution

The program successfully checks scholarship eligibility using conditional statements based on marks, attendance, and annual family income

CASE STUDY 2: EMPLOYEE p AND DEDUCTION ANALYSIS

1.Problem statement

To develop a Python program to calculate an employee's gross salary, deductions, and net salary.

2.Objectives

To calculate gross salary

To calculate salary deductions.

To find net salary.

To display the salary statement.

3.Assumption

- Basic salary, allowances, and overtime are entered by the user.
- Deduction rate is 10% of gross salary.
- Net salary equals gross salary minus deductions.

4. Algorithm

1.Start.

2.Read basic salary, allowances, and overtime.

3.Calculate gross salary.

4. Calculate 10% deduction.

5. Calculate net salary.

6. Display all amounts.

7. Stop.

5.Flow chart

START

|

v

Input Salary,

Allowance, OT

|

v

Calculate Gross

|

v

Calculate Deduction

|

v

Calculate Net Salary

|

v

Display Results

|

v

STOP

6.Pseudocode

BEGIN

INPUT basic, allowance, overtime

gross = basic + allowance + overtime

deduction = gross * 0.10

```
net = gross - deduction
PRINT gross, deduction, net
END
```

7.Python source code

```
basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowance: "))
overtime = float(input("Enter overtime amount: "))

gross = basic + allowance + overtime
deduction = gross * 0.10
net = gross - deduction

print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net)
```

8.Test cases

Basic	Allowance	Overtime	Net Salary
20000	3000	2000	22500
30000	5000	0	31500

9.Output screenshot

```
Enter basic salary: 60000
Enter allowance: 3000
Enter overtime amount: 2000
Gross Salary: 65000.0
Deduction: 6500.0
Net Salary: 58500.0
```

10.Conclusion

The program calculates gross salary, deductions, and net salary using arithmetic operators.

CASE STUDY 3: RETAIL BILLING WITH DISCOUNT RULES

1.Problem statement:

To develop a Python program to calculate the total price of purchased products and apply discounts based on the bill amount.

2. Objectives

To calculate the total bill.

To apply discounts.

To calculate the final amount.

To display billing details.

3.Assumptions

The user enters product price and quantity.

A 10% discount is applied when the total is at least Rs. 1,000.

No discount is given below Rs. 1,000.

4.algorithm

1.Start

2.Read price and quantity.

3.Calculate total amount.

4.Check whether the total is at least Rs. 1,000.

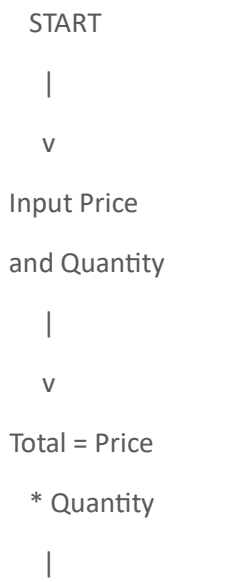
5 .Calculate discount.

6 .Calculate final bill.

7.Display the bill.

8.Stop.

5.Flow chart





6.Pseudocode

```
BEGIN
INPUT price, quantity
total = price * quantity

IF total >= 1000 THEN
   discount = total * 0.10
ELSE
   discount = 0
END IF

bill = total - discount
PRINT bill
END
```

7.Python source code

```
price = float(input("Enter product price: "))
quantity = int(input("Enter quantity: "))
```

```
if price >= 0 and quantity > 0:
    total = price * quantity

    if total >= 1000:
        discount = total * 0.10
    else:
        discount = 0

    bill = total - discount

    print("Total Amount:", total)
    print("Discount:", discount)
    print("Final Bill:", bill)
else:
    print("Invalid Input")
```

8.Test cases

Price	Quantity	Expected Final Bill
500	3	1350
200	2	400

9.Output screenshot

```
Enter product price: 500
Enter quantity: 3
Total Amount: 1500.0
Discount: 150.0
Final Bill: 1350.0
```

10.CONCLUSION

The program calculates retail bills and applies discounts using conditional statements.

CASE STUDY 4: ATTENDANCE RECOVERY COMPUTATION

1.PROBLEM STATEMENT

To develop a Python program to calculate student attendance percentage and determine examination eligibility.

2.OBJECTIVES

To calculate attendance percentage.

To check the minimum attendance requirement.

To display examination eligibility.

3.ASSUMPTION

The user enters total classes and attended classes.

Minimum required attendance is 75%.

Attended classes cannot exceed total classes.

4.ALGORITHM

1. Start.

2.Read total classes and attended classes.

3.Validate the input.

4.Calculate attendance percentage.

5.Check whether attendance is at least 75%.

6.Display the result.

7.Stop.

5.FLOW CHART

START

|

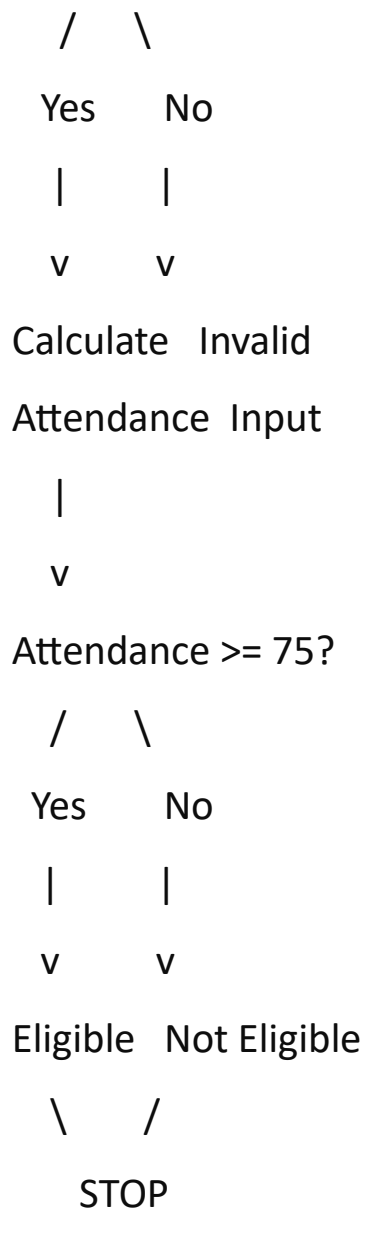
v

Input Classes

|

v

Valid Input?



6.PSEUDOCODE

```
BEGIN
INPUT total, attended

IF total > 0 AND attended >= 0
AND attended <= total THEN
   percentage = attended / total * 100
```

```
    IF percentage >= 75 THEN
        PRINT "Eligible"
    ELSE
        PRINT "Not Eligible"
    END IF
ELSE
    PRINT "Invalid Input"
END IF

END
```

7.PYTHON SOURCE CODE

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))

if total > 0 and 0 <= attended <= total:
    percentage = attended / total * 100

    print("Attendance:", percentage, "%")

    if percentage >= 75:
        print("Eligible for Examination")
    else:
        print("Not Eligible for Examination")
else:
```

```
    print("Invalid Input")
```

8.TEST CASES

Total Classes	Attended	Attendance	Result
100	80	80%	Eligible
100	60	60%	Not Eligible

9.OUTPUT SCREENSHOT

```
Enter total classes: 75
Enter attended classes: 45
Attendance: 60.0 %
Not Eligible for Examination
```

10.CONCLUSION

The program calculates attendance percentage and determines examination eligibility.

CASE STUDY 5: ATM TRANSACTION SEQUENCE SIMULATION

1.PROBLEM STATEMENT

To develop a Python program to simulate ATM transactions such as deposits, withdrawals, and balance inquiries.

2.OBJECTIVES

To perform deposits.

To process withdrawals.

To display account balance.

To validate transactions.

3.ASSUMPTION

Initial balance is Rs. 5,000.

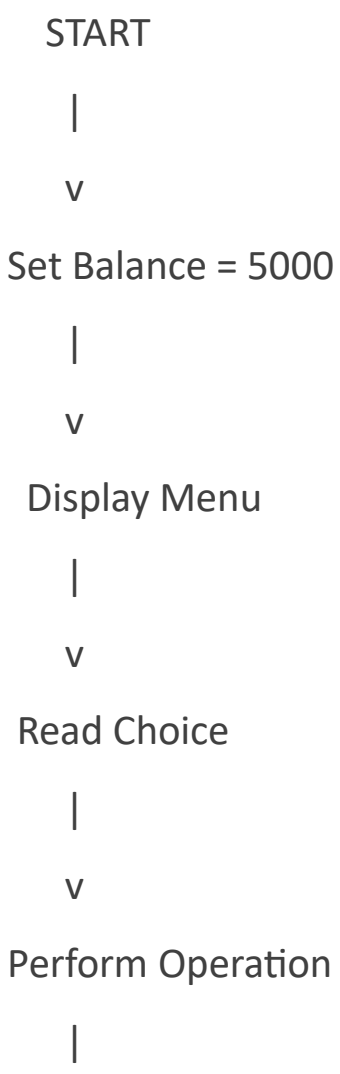
Deposits must be positive.

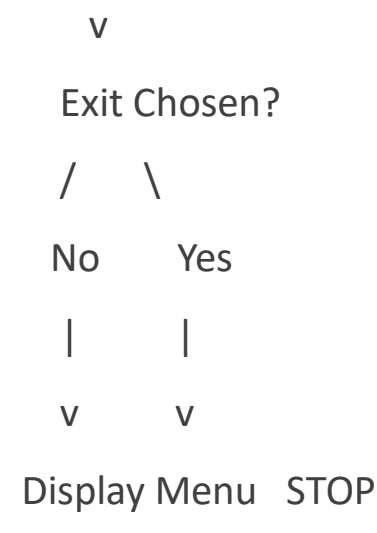
Withdrawals cannot exceed the available balance.

4.ALGORITHM

1. Start.
2. Set the initial balance to Rs. 5,000.
3. Display the ATM menu.
4. Read the user's choice.
5. Perform the selected operation.
6. Update the balance when necessary.
7. Repeat until the user chooses Exit.
8. Stop.

5.FLOW CHART





6.PSEUDOCODE

```
BEGIN
balance = 5000

WHILE TRUE
   DISPLAY ATM MENU
   INPUT choice

   IF choice = 1 THEN
      INPUT amount
      IF amount > 0 THEN
         balance = balance + amount
      END IF

   ELSE IF choice = 2 THEN
      INPUT amount
      IF amount > 0 AND amount <= balance THEN
```

```
            balance = balance - amount
        END IF

    ELSE IF choice = 3 THEN
        PRINT balance

    ELSE IF choice = 4 THEN
        BREAK
    END IF
END WHILE

END
```

7.PYTHON SOURCE CODE

```
balance = 5000

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
```

```
        amount = float(input("Enter deposit amount: "))

        if amount > 0:
            balance += amount
            print("Deposit Successful")
        else:
            print("Invalid Amount")

    elif choice == "2":
        amount = float(input("Enter withdrawal amount: "))

        if 0 < amount <= balance:
            balance -= amount
            print("Withdrawal Successful")
        else:
            print("Invalid Amount or Insufficient Balance")

    elif choice == "3":
        print("Available Balance:", balance)

    elif choice == "4":
        print("Thank You!")
        break
```

```
    else:
        print("Invalid Choice")
```

8.TEST CASES

Operation	Amount	Expected Result
Deposit	1000	Balance becomes 6000
Withdraw	2000	Balance becomes 3000
Withdraw	6000	Transaction rejected
Check Balance	—	Displays current balance

9.OUTPUT SCREENSHOT

```
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 1
Enter deposit amount: 1
Deposit Successful
```

10. CONCLUSION

The program simulates basic ATM operations using loops, conditional statements, and variables.

CASE STUDY 6: ADVANCED NUMERIC PATTERN GENERATOR

1.PROBLEM STATEMENT

To develop a Python program to generate a numerical pattern using nested loops.

2.OBJECTIVES

To generate number patterns.

To understand nested loops.

To display formatted output

3.assumptions

The user enters the number of rows.

The number of rows must be positive.

Each row displays numbers from 1 up to its row number.

4.ALGORITHM

1.Start.

2.Read the number of rows.

3.Check whether the number is positive.

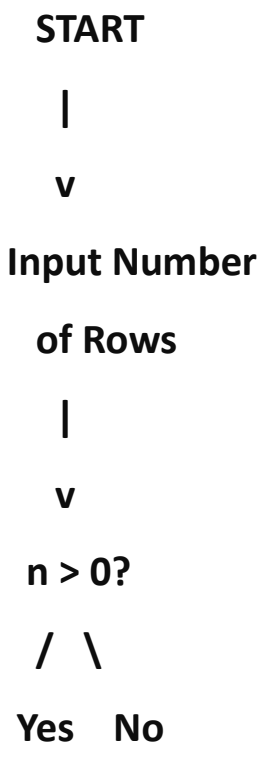
4.Use an outer loop for rows.

5.Use an inner loop to print numbers.

6.Move to the next line after each row.

7.Stop.

5.FLOW CHART



| |

v v

Outer Loop Error

|

v

Inner Loop

|

v

Print Pattern

|

v

STOP

6.PSEUDOCODE

```
BEGIN
INPUT n

IF n > 0 THEN
   FOR i = 1 TO n
      FOR j = 1 TO i
         PRINT j
      END FOR
      PRINT NEW LINE
   END FOR
ELSE
```

```
    PRINT "Invalid Input"
END IF


END
```

7.PYTHON SOURCE CODE

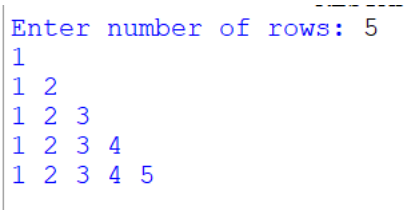
```
n = int(input("Enter number of rows: "))


if n > 0:
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
else:
    print("Invalid Input")
```

8.TEST CASES

Input	Expected Output
3	Three rows of numbers
5	Five rows of numbers
0	Invalid Input

9.OUTPUT SCREENSHOT



10.CONCLUSION

The program generates a numerical pattern using nested loops.

CASE SYUDY 7: PASSWORD POLICY VALIDATION ENGINE

1. PROBLEM STATEMENT

TO develop a Python program to validate a password based on its minimum length.

2.OBJECTIVES

.To accept a password from the user.

To check password length.

To display whether the password meets the minimum length requrirement

3.ASSUMPTION

- The minimum password length is 8 characters.
- This basic program checks length only.
- Other password rules are not checked

4.algorithm

1. Start.

2.Read the password.

3.Calculate its length.

4.Check whether the length is at least 8.

5.Display the result.

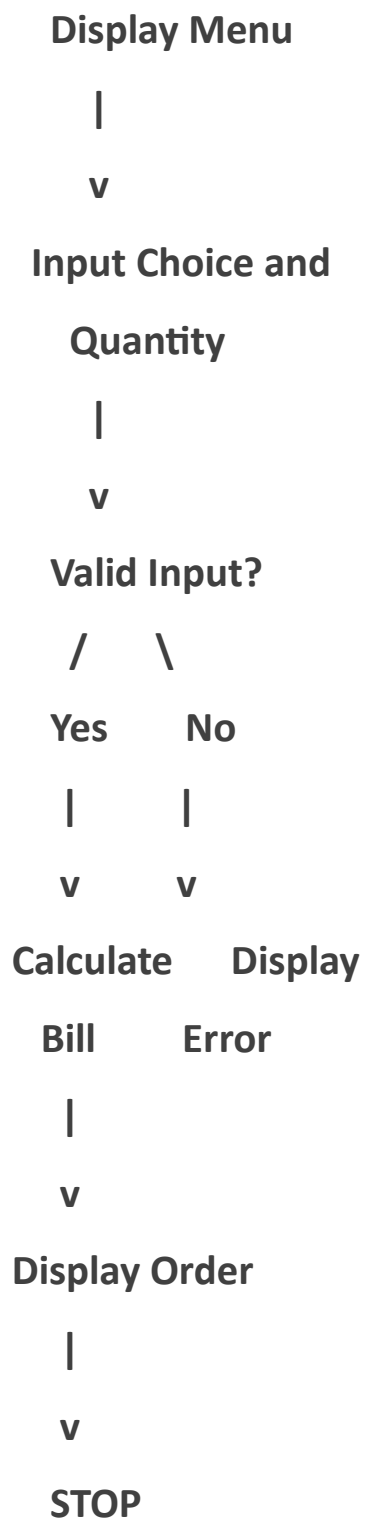
6.Stop.

5. Flowchart

START

|

v



6. Pseudocode

```
BEGIN
menu = [Idli, Dosa, Meals]
prices = [30, 50, 80]
```

```
DISPLAY menu
INPUT choice, quantity

IF choice is between 1 and 3
AND quantity > 0 THEN
   total = prices[choice - 1] * quantity
   PRINT menu[choice - 1]
   PRINT quantity
   PRINT total
ELSE
   PRINT "Invalid Input"
END IF

END
```

7. Python Source Code

```
menu = ["Idli", "Dosa", "Meals"]
prices = [30, 50, 80]

print("1. Idli - Rs.30")
print("2. Dosa - Rs.50")
print("3. Meals - Rs.80")

choice = int(input("Choose food item (1-3): "))
```

```
quantity = int(input("Enter quantity: "))

if 1 <= choice <= 3 and quantity > 0:
    total = prices[choice - 1] * quantity

    print("Food Item:", menu[choice - 1])
    print("Quantity:", quantity)
    print("Total Bill: Rs.", total)
else:
    print("Invalid Choice or Quantity")
```

8.Test Cases

Choice	Quantity	Expected Total
1	2	Rs. 60
2	3	Rs. 150
3	2	Rs. 160
5	2	Invalid Input

9. Output Screenshot

```
Enter password: keerthi
Password Must Contain At Least 8 Characters
```

10.conclution

The program manages a basic canteen order and calculates the bill using lists, indexing, and conditional statements.

CASE STUDY 8: STRING-BASED EXAMINATION ANSWER EVALUATION1.

1.problem statement

To develop a Python program to compare student answers with an answer key and calculate examination marks.

2. objectives

- **To store student answers.**
- **To compare answers with the key.**
- **To count correct and incorrect answers.**
- **To calculate total marks.**

3. assumption

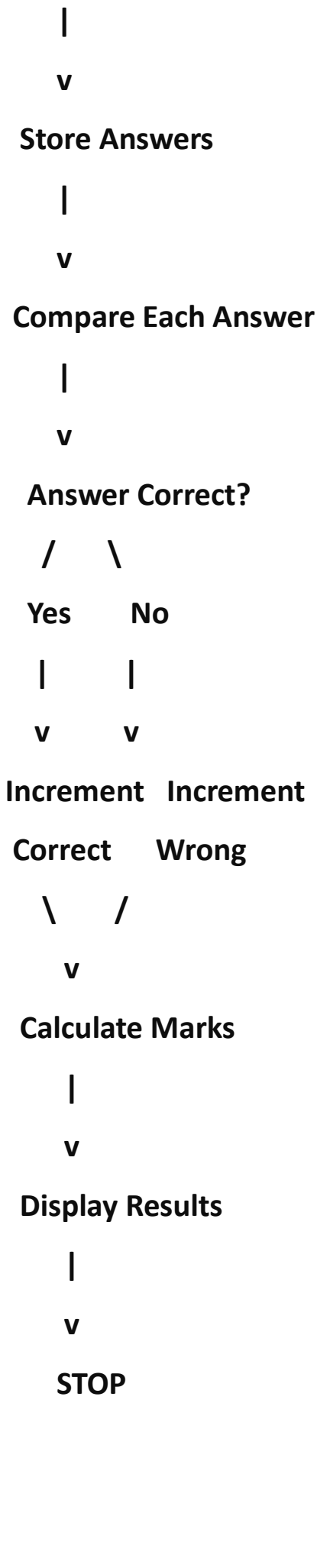
- **There are five questions.**
- **Each correct answer carries 2 marks.**
- **Incorrect answers carry zero marks.**
- **No negative marking is applied.**

4. algorithm

1. **Start.**
2. **Store student answers and the answer key.**
3. **Set correct and wrong counts to zero.**
4. **Compare each answer with the corresponding key.**
5. **Update the counts.**
6. **Calculate total marks.**
7. **Display the results.**
8. **Stop.**

5. flow chart

START


Store Answers
Compare Each Answer
Answer Correct?
Yes
No
Increment
Correct
Increment
Wrong
Calculate Marks
Display Results
STOP

6. pseudocode

```
BEGIN
answers = [A, B, C, D, A]
key = [A, B, D, D, A]

correct = 0
wrong = 0

FOR each answer
   IF answer matches key THEN
      correct = correct + 1
   ELSE
      wrong = wrong + 1
   END IF
END FOR

marks = correct * 2
PRINT correct, wrong, marks
END
```

7. python source code

```
answers = ["A", "B", "C", "D", "A"]
key = ["A", "B", "D", "D", "A"]

correct = 0
```

```
wrong = 0


for i in range(len(key)):
    if answers[i] == key[i]:
        correct += 1
    else:
        wrong += 1


marks = correct * 2


print("Correct Answers:", correct)
print("Wrong Answers:", wrong)
print("Total Marks:", marks)
```

8. test cases

Correct Answers	Wrong Answers	Total Marks
4	1	8
5	0	10
3	2	6

9. out put screen shot

```
Correct Answers: 4
Wrong Answers: 1
Total Marks: 8
```

10.Conclusion

The program evaluates examination answers using lists, loops, and conditional statements.

CASE STUDY 9: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

To develop a Python program to analyse student marks, calculate totals and averages, and identify academic performance.

2. Objectives

- **To store subject marks in a list.**
- **To calculate total marks.**
- **To calculate average marks.**
- **To classify performance using conditions.**

3. Assumptions

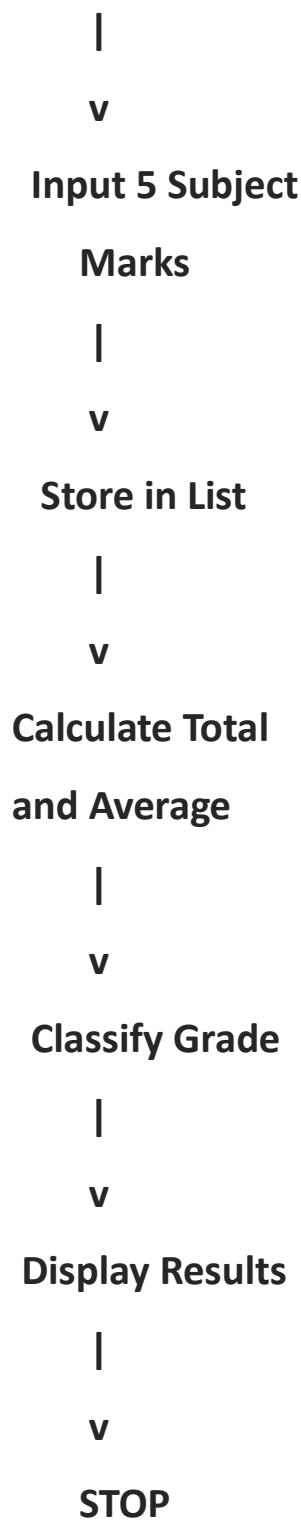
- **The student has five subjects.**
- **Each subject has a maximum of 100 marks.**
- **Grade classification is based on average marks.**

4. Algorithm

1. **Start.**
2. **Create an empty list.**
3. **Read marks for five subjects.**
4. **Store marks in the list.**
5. **Calculate total and average.**
6. **Classify the result.**
7. **Display the marks and result.**
8. **Stop.**

5. Flowchart

START



6. Pseudocode

```
BEGIN
marks = EMPTY LIST
```

```
REPEAT 5 TIMES
    INPUT mark
    ADD mark TO marks
END REPEAT

total = SUM(marks)
average = total / 5

IF average >= 75 THEN
    grade = A
ELSE IF average >= 60 THEN
    grade = B
ELSE IF average >= 40 THEN
    grade = C
ELSE
    grade = Fail
END IF

PRINT total, average, grade
END
```

7. Python Source Code

```
marks = []

for i in range(5):
```

```
    mark = int(input("Enter subject marks: "))

    if 0 <= mark <= 100:
        marks.append(mark)
    else:
        print("Invalid Marks")
        break

if len(marks) == 5:
    total = sum(marks)
    average = total / 5

    print("Subject Marks:", marks)
    print("Total Marks:", total)
    print("Average Marks:", average)

    if average >= 75:
        print("Grade: A")
    elif average >= 60:
        print("Grade: B")
    elif average >= 40:
        print("Grade: C")
    else:
        print("Grade: Fail")
```

8. Test Cases

Marks in Five Subjects	Average	Expected Grade
80, 80, 80, 80, 80	80	A
60, 60, 60, 60, 60	60	B
40, 40, 40, 40, 40	40	C
20, 20, 20, 20, 20	20	Fail

9. Output Screenshot

```
Enter subject marks: 98
Enter subject marks: 89
Enter subject marks: 97
Enter subject marks: 67
Enter subject marks: 85
Subject Marks: [98, 89, 97, 67, 85]
Total Marks: 436
Average Marks: 87.2
Grade: A
```

10. Conclusion

The program analyses student marks using lists, loops, arithmetic operators, and conditional statements.

CASE STUDY 10: SMART CANTEEN ORDER AND STOCK MANAGEMENT

1. Problem Statement

To develop a Python program to accept food orders, calculate bills, and display the selected food item and quantity.

2. Objectives

- **To display the food menu.**
- **To accept customer orders.**
- **To calculate the total bill.**
- **To validate the selected item and quantity.**

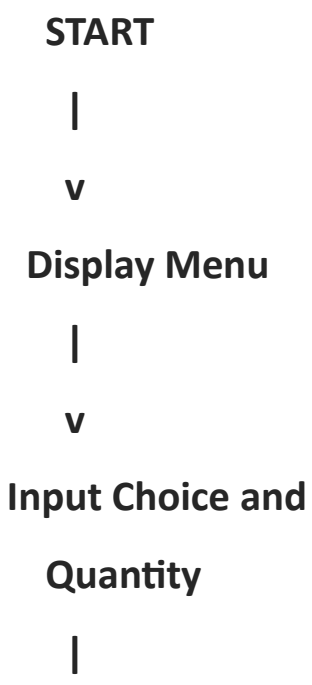
3. Assumptions

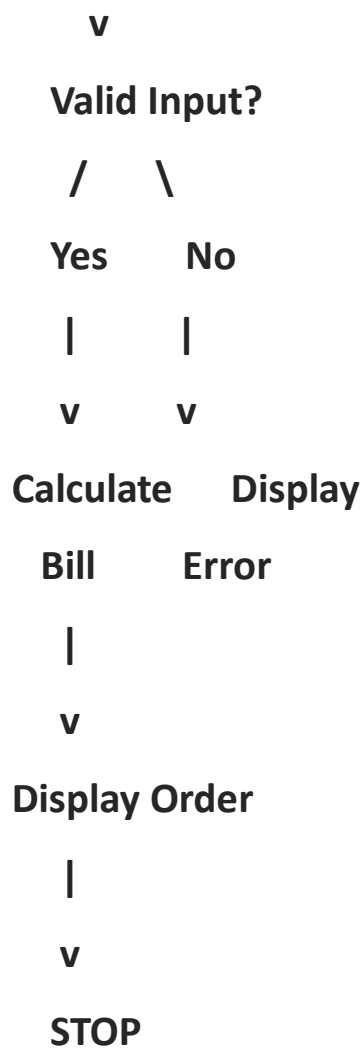
- The menu contains Idli, Dosa, and Meals.
- Their prices are Rs. 30, Rs. 50, and Rs. 80.
- The customer selects one item per order.
- Quantity must be positive.

4. ALGORITHM

1. Start.
2. Create lists of food items and prices.
3. Display the menu.
4. Read the customer's choice and quantity.
5. Validate the inputs.
6. Calculate the total bill.
7. Display the order details.
8. Stop.

5. Flowchart





6. Pseudocode

```
BEGIN
menu = [Idli, Dosa, Meals]
prices = [30, 50, 80]

DISPLAY menu
INPUT choice, quantity

IF choice is between 1 and 3
AND quantity > 0 THEN
```

```
    total = prices[choice - 1] * quantity
    PRINT menu[choice - 1]
    PRINT quantity
    PRINT total
ELSE
    PRINT "Invalid Input"
END IF

END
```

7. Python Source Code

```
menu = ["Idli", "Dosa", "Meals"]
prices = [30, 50, 80]

print("1. Idli - Rs.30")
print("2. Dosa - Rs.50")
print("3. Meals - Rs.80")

choice = int(input("Choose food item (1-3): "))
quantity = int(input("Enter quantity: "))

if 1 <= choice <= 3 and quantity > 0:
    total = prices[choice - 1] * quantity

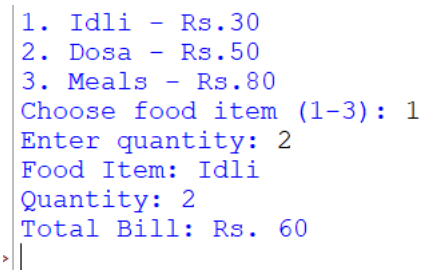
    print("Food Item:", menu[choice - 1])
```

```
    print("Quantity:", quantity)
    print("Total Bill: Rs.", total)
else:
    print("Invalid Choice or Quantity")
```

8. Test Cases

Choice	Quantity	Expected Total
1	2	Rs. 60
2	3	Rs. 150
3	2	Rs. 160
5	2	Invalid Input

9. Output Screenshot



10. Conclusion

The program manages a basic canteen order and calculates the bill using lists, indexing, and conditional statements.
